



UNIVERSIDAD TÉCNICA
FEDERICO SANTA MARÍA

Profesor Oscar Reyes
Pruebas de Software (INF331)



Departamento de Informática
Universidad Técnica Federico Santa María

Proyecto de Pruebas de software

Entrega 3, Equipo 4

Joaquín De Ferrari, Javier Maturana y Rodrigo Tallar

Github:

<https://github.com/Pruebas-intensas/pruebas-intensas.github.io>

<https://github.com/Pruebas-intensas/Proyecto-Back>



Índice

1. Introducción.....	3
2. Alcances de la herramienta.....	3
3. Descripción del trabajo realizado.....	3
4. Problemas encontrados y soluciones.....	5



1. Introducción

El presente trabajo trató sobre la continuación del desarrollo de una aplicación web de remates en línea, donde clientes pueden realizar pujas y un administrador puede agregar productos/subastas.

Para esta entrega el foco era realizar pruebas de interfaz por medio de la herramienta selenium. Selenium es un software de automatización de pruebas que permite programar interacciones con un navegador para realizar pruebas funcionales de la interfaz, mediante la simulación de acciones del usuario.

2. Alcances de la herramienta

Se busca implementar tests con Selenium para probar distintas funcionalidades importantes de la interfaz del front end. En este caso se decidió probar 5 funcionalidades: iniciar sesión en la página, registrarse, crear un producto (o subasta) nuevo, eliminar un producto y hacer una puja, dado que estas son tareas centrales en el funcionamiento de software.

La idea es crear estos tests y hacer que estos se ejecuten de manera automática mediante github actions cuando se complete una pull request sobre la rama main. Además, los tests tendrán comentarios a lo largo de la ejecución que nos indicarán en qué parte fallan.

3. Descripción del trabajo realizado

Al igual que en los trabajos anteriores, se designaron una serie de tareas mediante jira, que incluyeron definir qué pruebas realizar, crear el código para llevar a cabo las pruebas e integrar en github actions. También se utilizaron las notificaciones de Slack para cambios en Github y tareas completadas o designadas en Jira. Al mismo tiempo, se utilizó Gitflow como flujo de trabajo, creando ramas para features, realizando pull requests a una rama dev, y acumulando algunas features antes de hacer una pull request a la rama main. También se utilizaron ramas y pull requests para bugfixes y hotfixes en caso de ser necesario.

Para la ejecución de los test no se usó ninguna otra herramienta de automatización, como por ejemplo Mocha. Se escribió el código directamente en un archivo TypeScript que se compila a Javascript para ejecutar las pruebas. Para obtener los resultados finales se hicieron “asserts” caseros, por ejemplo, si esperábamos que un



resultado fuera 5 y el resultado era 5 retornamos True, sino, False. Luego paseábamos estos resultados y se mostraba en consola cuantos True vs. tests totales se tenían para obtener el porcentaje de tests pasando.

```
import { WebDriver, Builder, By, until } from 'selenium-webdriver';
import { Options } from 'selenium-webdriver/chrome';
import { environment } from '../environments/environment';

async function loginTest(driver: WebDriver) {
  try {
    console.log("\nIntentando ingresar a la página...", environment.url_front);
    await driver.get(environment.url_front);
    await driver.sleep(100);
    await driver.manage().window().maximize();

    console.log("Intentando ingresar usuario y contraseña...");
    await driver.wait(until.elementLocated(By.id('exampleInputEmail')), 10000).sendKeys('user@gmail.com');
    await driver.wait(until.elementLocated(By.id('exampleInputPassword')), 10000).sendKeys('123');
    //press the button called myBtn
    const boton = await driver.wait(until.elementLocated(By.id('myBtn')), 10000);
    await boton.click();
    // check if the element with id "userDropdown" is present
    console.log("Esperando inicio de sesión..");
    await driver.wait(until.elementLocated(By.id('userDropdown')), 10000);
    const currentUrl = await driver.getCurrentUrl();
    return currentUrl === environment.url_front + '/home';
  } catch (error) {
    console.error('Error:', error);
    return false;
  }
}
```

Código para el test del login. Se puede ver que se entra a la pestaña del login, se ingresan los datos de una cuenta creada y luego se chequea si la nueva URL de la página corresponde a la de cuando un usuario está logueado.

```
Test with selenium
42 Intentando crear producto...
43 Intentando ingresar usuario y contraseña...
44 Esperando inicio de sesión..
45 crearProductoTest exitoso
46
47 Intentando ingresar a la página... https://pruebas-intensas.github.io
48 Intentando ingresar usuario y contraseña...
49 Esperando inicio de sesión..
50 Intentando ingresar a la página de productos...
51 Esperando página de productos...
52 Intentando ingresar a la página de agregar productos...
53 Esperando modal de agregar productos...
54 Intentando ingresar nombre...
55 Intentando ingresar precio...
56 Intentando ingresar descripción...
57 Intentando ingresar fecha...
58 Intentando crear producto...
59 Intentando ingresar usuario y contraseña...
60 Esperando inicio de sesión..
61 Intentando ingresar a la página de productos...
62 Intentando eliminar producto...
63 eliminarProductoTest exitoso
64
65 Intentando ingresar a la página... https://pruebas-intensas.github.io
66 Intentando ingresar usuario y contraseña...
67 Esperando inicio de sesión..
68 Intentando ingresar a la página del producto...
69 Esperando página del producto...
70 Intentando ingresar puja...
71 Aceptando puja...
72 Chequeando puja realizada...
73 realizarPujaTest exitoso
74
75 5/5 tests exitosos
```

Output de la consola de Github actions, mostrando algunos de los tests realizados, junto a cada uno de los pasos realizados a lo largo de su ejecución. En este ejemplo en particular se aprobaron los 5 tests tras el último push a la rama main.



4. Problemas encontrados y soluciones

Con el uso de selenium nos surgieron muchísimos problemas. Primero nos ocurrió que selenium no encontraba los elementos que intentábamos buscar por id, tratamos usando la función wait de selenium y también usando sleeps y parecía que ninguno lo solucionaba. También pasaba que cuando corríamos las pruebas de forma local estas pasaban, pero cuando lo subíamos al github ni siquiera se corrían las pruebas, esto se solucionó usando chrome como browser headless. Al hacer este cambio ahora se ejecutaban las pruebas pero ninguna pasaba. Al final descubrimos que ciertas partes hacían conflicto al buscar elementos cuyos id's se generaban dinámicamente por lo que tuvimos que repensar el cómo asignar los id's.